Canvas Line Sender 코드 초상세 해설서

이벤트 처리 블록 단위 설명 포함 · Dart/Flutter 문법부터 좌표 변환, 스트로크, JSON/TCP 전송까지

이 문서의 목표 중학생도 따라올 수 있도록 쉽게 설명하되, 실제 코드를 고칠 수 있을 정도로 충분히 자세하게 풀어쓴다. 특히 이전 설명에서 빠졌던 _onPointerDown 내부의 조건문 블록, setState 안에서 상태를 어떻게 바꾸는지, Continue 기능이 실제로 어떤 식으로 동작하는지까지 넣었다.

1. 이 프로그램을 한 문장으로 말하면?

사용자가 캔버스 위에 선이나 도형을 그리면, 프로그램이 그 모양을 점들의 리스트로 바꾸고, 필요하면 mm 단위로 환산하고, Z 값까지 붙여서 JSON 으로 만들어 TCP 소켓으로 서버에 보내는 Flutter 앱이다.

핵심 흐름 마우스 누름 → 상태 초기화 → 움직이며 점 생성 → 샘플링 → 좌표 변환 → 표에 표시 → JSON 생성 → TCP 전송

2. 큰 구조 먼저 보기

부분	역할
main()	앱 실행 시작점
CanvasApp	MaterialApp 설정
CanvasHomePage	실제 화면을 가지는 StatefulWidget
_CanvasHomePageState	거의 모든 로직이 들어 있는 핵심 클래스
LinePainter	캔버스 위에 선, 점, 선택 박스를 그림
Stroke	한 개의 선 묶음(path, samples, zValues) 저장
WebViewerPage	Onshape 같은 웹페이지를 앱 안에서 띄움

3. Dart 문법 기초

import

다른 파일이나 라이브러리의 기능을 가져온다. 예를 들어 material.dart 를 가져오면 버튼, 텍스트필드, Scaffold 같은 Flutter UI 부품을 쓸 수 있다.

class

데이터와 기능을 하나로 묶는 설계도다. CanvasApp, Stroke, WebViewerPage 가 모두 class 다.

extends

기존 클래스의 성질을 이어받는다는 뜻이다. StatelessWidget, StatefulWidget, CustomPainter 를 상속해 Flutter 규칙에 맞게 동작한다.

final

한 번 정해진 뒤 다시 바꾸지 않는 변수다. 값은 나중에 정할 수 있지만 한 번 정해지면 재할당이 안 된다.

const

컴파일 시점에 완전히 고정되는 값이다. 버튼 텍스트, 기본 숫자처럼 절대 안 바뀌는 값에 자주 쓴다.

?

nullable 문법이다. Offset?는 값이 없을 수도 있다는 뜻이다. 그래서 사용 전에 null 인지 확인해야 한다.

??

왼쪽이 null 이면 오른쪽 값을 대신 쓴다. last ?? startPos 는 last 가 없으면 startPos 를 쓴다는 뜻이다.

async / await

시간이 걸리는 작업을 기다릴 때 쓴다. 소켓 연결, 이미지 불러오기, 파일 선택 등에서 사용한다.

setState

상태가 바뀌었다는 사실을 Flutter 에게 알려 화면을 다시 그리게 한다.

List<Offset>

Offset 점들을 담는 리스트다. 선의 경로나 샘플 점들을 저장한다.

4. Flutter 화면 구조

build()는 화면을 조립하는 함수다. 이 코드에서는 Scaffold 안에 AppBar 와 body 가 들어 있고, body 는 다시 캔버스 영역과 컨트롤 패널로 나뉜다. 가로가 넓으면 좌우 배치, 세로가 긴 화면이면 위아래 배치가 되도록 LayoutBuilder 와 MediaQuery 를 사용한다.

위젯	쉽게 말하면
Scaffold	앱의 기본 뼈대
AppBar	맨 위 제목 줄
LayoutBuilder	현재 화면 크기를 보고 배치 전략 결정
Stack	위젯을 겹쳐 놓기
Listener	마우스/포인터 입력 감지
CustomPaint	직접 선과 점을 그림
TextField	사용자 입력창
ElevatedButton	눌러서 동작 실행
DataTable	점 좌표와 Z 값을 표 형태로 표시

5. 상태 변수들을 묶음별로 이해하기

그리기 입력 상태

- _p0, _p1 은 시작점과 끝점이다.
- _rawPoints 는 펜 모드에서 사용자가 실제로 지나간 원본 점들이다.
- _isDrawing 은 지금 그리고 있는 중인지 알려준다.

좌표/샘플 상태

- _pointsPx 는 캔버스 좌표의 샘플 점들이다.
- _pointsImage 는 프레임 기준 로컬 좌표다.
- _pointsMm 는 mm 로 변환된 점들이다.
- _deltasMm 는 연속 점 사이 차이 벡터다.

Z 관련

- _currentZValues 는 아직 stroke 로 확정되지 않은 현재 도형의 Z 값들이다.
- 각 Stroke 도 자기만의 zValues 를 가진다.

스트로크 관리

- _strokes 는 완성된 선들의 모음이다.
- _selectedStrokeIndex 는 현재 선택된 선 번호다.
- _appendPen 은 Continue 기능이 켜졌는지 나타낸다.

선택/이동/편집

- _selectMode 는 사각형 드래그로 선택하는 모드다.
- _isMoving 은 우클릭 드래그로 스트로크를 이동 중인지 나타낸다.
- _isEditingPoint 는 Shift+드래그로 점 편집 중인지 나타낸다.

배경/프레임

- _imageFile 은 배경 이미지 파일이다.
- _frameRect 는 실제 화면에서 그림을 그릴 수 있는 사각형이다.
- _userFrameMmSize 와 _userFramePxSize 는 사용자가 지정한 기준 크기다.

통신

- _socket 은 서버와 연결된 소켓 객체다.
- _connectedHost, _connectedPort 는 현재 연결 대상을 기억한다.
- _status 는 화면에 보여줄 상태 문자열이다.

6. Stroke 가 왜 중요한가?

Stroke 는 한 개의 선 덩어리를 뜻한다. 이 클래스에는 path, samples, zValues 가 들어 있다.

이름	의미	왜 따로 필요한가
path	원래 경로 점들	보이는 선 자체를 표현하기 좋다
samples	일정 간격으로 정리된 점들	전송이나 표 표시용으로 쓰기 좋다
zValues	각 샘플 점의 Z 값	x,y 와 짝을 이루어 3 차원 포인트를 만든다

중요 path 와 samples 가 완전히 같은 리스트가 아닐 수 있다. pen 으로 쭉 그린 원본 경로는 들쭉날쭉할 수 있고, samples 는 일정 간격으로 다시 뽑아낸 점들이기 때문이다.

7. 이벤트 처리 전체 흐름

1. 마우스를 누르면 _onPointerDown 이 실행된다.
2. 마우스를 움직이면 _onPointerMove 가 계속 실행된다.
3. 마우스를 떼면 _onPointerUp 이 실행된다.
4. 이 과정에서 _rebuildPoints 가 여러 번 호출되며 현재 점 리스트를 계속 다시 만든다.
5. 마지막에 완성된 점들은 _strokes 에 저장되고, 필요하면 _sendJson 으로 서버에 보낸다.

8. 핵심: _onPointerDown 의 이벤트 처리 블록 설명

사용자가 마우스를 누른 순간 실행되는 코드다. 이 함수는 “지금부터 어떤 방식으로 그리기를 시작할지”를 정한다. 아래 블록이 특히 중요하다.

```
setState(() {
  _isDrawing = true;

  if (_mode == DrawMode.line) {
    final startPos = _clampToFrame(event.localPosition);
    if (_appendPen && _strokes.isNotEmpty) {
      _appendStrokeIndex = _strokes.length - 1;
      final last =
          _strokes.last.path.isNotEmpty ? _strokes.last.path.last : null;
      _p0 = last ?? startPos;
      _p1 = startPos;
    } else {
      _appendStrokeIndex = null;
      _p0 = startPos;
      _p1 = startPos;
    }
  } else {
    if (_mode == DrawMode.pen && _appendPen && _strokes.isNotEmpty) {
      _appendStrokeIndex = _strokes.length - 1;
      final last =
```

```
        _strokes.last.path.isNotEmpty ? _strokes.last.path.last : null;
      if (last != null) {
        _rawPoints = <Offset>[last, _clampToFrame(event.localPosition)];
        _p0 = last;
      } else {
        final pos = _clampToFrame(event.localPosition);
        _rawPoints = <Offset>[pos];
        _p0 = pos;
      }
    } else {
      _appendStrokeIndex = null;
      final pos = _clampToFrame(event.localPosition);
      _rawPoints = <Offset>[pos];
      _p0 = pos;
    }
    _p1 = _clampToFrame(event.localPosition);
  }

  _rebuildPoints();
});
```

8-1. setState(() { ... })

이 블록 안에서 바뀌는 값들은 화면에도 바로 반영된다. Flutter 는 setState 가 호출되면 build 와 painter 를 다시 돌려서 선, 점, 표를 최신 상태로 그린다.

8-2. _isDrawing = true

이 값은 "지금 사용자가 실제로 그림을 그리고 있다"는 신호다. 나중에 _onPointerMove 에서 이 값이 false 면 이동 이벤트를 무시한다. 그래서 마우스를 그냥 움직이는 것과 실제 그리기를 구분할 수 있다.

8-3. if (_mode == DrawMode.line)

line 모드는 시작점과 끝점만 있으면 된다. 반면 pen 은 지나간 점 전체가 필요하다. 그래서 line 과 나머지 모드를 크게 둘로 나눈다.

8-4. startPos = _clampToFrame(event.localPosition)

event.localPosition 은 클릭한 화면 좌표다. 하지만 프레임 밖 좌표가 들어오면 계산이 꼬일 수 있으므로 _clampToFrame 으로 프레임 안쪽으로 강제 제한한다.

예시 프레임 오른쪽 끝이 x=800 인데 사용자가 x=850 을 눌렀다면 850 대신 800 으로 보정한다.

8-5. line 모드 + Continue 켜짐 + 기존 stroke 존재

조건식은 _appendPen && _strokes.isNotEmpty 이다. 뜻은 이어그리기가 켜져 있고, 이어 붙일 기존 선도 하나 이상 있어야 한다는 뜻이다.

- _appendStrokeIndex = _strokes.length - 1 : 마지막 stroke 번호를 기억한다.
- _strokes.last.path.last : 마지막 stroke 의 마지막 점을 가져온다.
- _p0 = last ?? startPos : 이전 선의 끝점이 있으면 거기서 시작하고, 없으면 현재 클릭 위치에서 시작한다.
- _p1 = startPos : 새 직선의 끝 후보를 현재 클릭 위치로 둔다.

왜 이렇게 하나? Continue 는 “새로운 선을 따로 만드는 것”이 아니라 “마지막 선 뒤에 이어 붙이는 것”이 목적이다. 그래서 시작점이 이전 선의 끝점이 된다.

8-6. line 모드 + 일반 새 선 시작

이어그리기가 아니면 _appendStrokeIndex 를 null 로 비워 둔다. 그리고 _p0 와 _p1 을 둘 다 현재 클릭 위치로 둔다. 처음에는 길이 0 인 선으로 시작하고, 이후 마우스를 움직일 때 _p1 만 바뀌면서 선이 길어진다.

8-7. else 블록: pen / circle / rect 처리

여기서는 line 이 아닌 모든 모드를 처리한다. 하지만 실제로 내부를 보면 pen 이어그리기와 일반 시작을 더 세밀하게 나눈다. circle, rect 는 _p0, _p1 만 있으면 되므로 pen 처럼 rawPoints 를 많이 쌓지 않는다. 현재 코드에서는 else 블록 구조 안에 pen 용 초기화가 중심이 된다.

8-8. pen 모드 + Continue 켜짐 + 기존 stroke 존재

이 경우는 선 하나를 계속 이어서 그리는 느낌을 만들기 위한 부분이다.

- _appendStrokeIndex = _strokes.length - 1 : 마지막 stroke 에 이어 붙이겠다고 표시한다.
- last 를 가져와서 이전 stroke 의 마지막 점을 확인한다.
- last 가 있으면 _rawPoints = [last, 현재위치] 로 시작한다. 즉 새로운 rawPoints 리스트 맨 앞에 이전 끝점을 넣는다.
- _p0 = last : 시작점은 이전 선 끝점이다.
- last 가 없으면 현재 위치 하나만 rawPoints 에 넣고 거기서 시작한다.

중요한 느낌 pen Continue 는 “마우스를 누른 순간부터 이미 한 점이 연결되어 있는 상태”를 만들기 위해 rawPoints 리스트에 이전 점을 넣어 둔다.

8-9. pen 모드 일반 시작

새로운 펜 선을 그리는 경우다. _appendStrokeIndex 를 null 로 하고, 현재 위치를 하나 담은 _rawPoints 를 만든다. 그리고 _p0 도 그 위치로 둔다. 이후 마우스를 움직일 때마다 _rawPoints 에 점이 계속 추가된다.

8-10. _p1 = _clampToFrame(event.localPosition)

line 이 아닌 경우에도 현재 포인터 위치를 _p1 에 기록해 둔다. 그래야 circle, rect 같은 도형은 시작점과 현재 점을 이용해 크기를 계산할 수 있고, pen 도 마지막 위치를 공통적으로 추적할 수 있다.

8-11. 마지막 한 줄: _rebuildPoints()

이 함수는 현재 상태를 바탕으로 실제 점 리스트를 다시 만든다. 이 코드 전체에서 가장 중요한 함수 중 하나다. 왜냐하면 화면에 보일 점, 표에 들어갈 점, mm 로 바뀐 점, Z 와 짝지을 점이 전부 여기서 다시 정리되기 때문이다.

한 문장 요약 _onPointerDown 은 시작 조건을 세팅하고, _rebuildPoints 는 그 조건을 실제 점 데이터로 변환한다.

9. _onPointerMove 는 무엇을 하나?

마우스를 움직일 때마다 호출된다. 그런데 상황별로 해야 할 일이 다르기 때문에 맨 위에서 먼저 분기한다.

상황	동작
_isMoving == true	스트로크 전체를 드래그 이동
_isEditingPoint == true	선 위의 한 점만 편집
_isSelecting == true	선택 사각형 크기 갱신
_isDrawing == false	아무것도 안 함
_mode == line	_p1 만 바꾸고 선 길이 갱신
그 외 pen	현재 위치를 _rawPoints 에 추가

10. _onPointerUp 은 왜 복잡한가?

마우스를 떼는 순간은 "임시 상태를 실제 stroke 로 확정하는 시점"이다. 그래서 이동 종료, 점 편집 종료, 선택 종료, 실제 그리기 종료가 모두 여기로 모인다.

6. 이동 중이면 _endMove()만 하고 끝낸다.
7. 점 편집 중이면 _endEditPoint()를 부른다.
8. 선택 중이면 선택 사각형을 없애고 어떤 stroke 를 고를지 계산한다.
9. 실제 그리기 중이었다면 마지막 위치를 반영하고 _rebuildPoints()를 다시 호출한다.
10. _pointsPx 가 비어 있지 않으면 Stroke 객체를 새로 만들거나 기존 stroke 에 append 한다.

11. _rebuildPoints 를 진짜 자세히 이해하기

이 함수는 현재 모드에 따라 점 생성 방식이 달라진다.

모드	점 생성 방식
line	_p0 와 _p1 사이를 일정 간격으로 분할
circle	원 둘레를 일정 간격으로 샘플링
rect	사각형 외곽선을 선분 단위로 샘플링
pen	rawPoints 를 일정 간격으로 다시 뽑아 재샘플링

11. 먼저 현재 모드에 따라 pointsImage 를 만든다.
12. 그 다음 image 좌표를 canvas 좌표로 바꿔 _pointsPx 를 만든다.
13. pointsImage 가 비어 있지 않으면 _applyTransform 으로 mm 좌표를 만든다.
14. _buildDeltas 로 인접 점 사이 차이를 만든다.
15. _currentZValues 길이를 점 개수와 맞춘다.

왜 재샘플링이 필요할까? 사람이 마우스로 그린 선은 점 간격이 일정하지 않다. 그런데 로봇이나 외부 장치에 보내려면 일정 간격의 점이 더 다루기 쉽다.

12. 좌표계가 3 개나 있는 이유

좌표계	뜻
Canvas 좌표	실제 화면에서 그려지는 위치
Image 좌표	프레임 내부 기준 좌표. 배경 이미지나 사용자 지정 프레임 기준
mm 좌표	실제 길이 단위로 변환된 좌표

_canvasToImageClamped 는 화면 좌표를 프레임 기준 좌표로 바꾸고, _imageToCanvas 는 반대로 바꾼다. _applyTransform 은 그 image 좌표에 mm 비율을 곱해서 실제 단위처럼 사용하게 만든다.

13. 샘플링 함수들

_resamplePolyline

원본 펜 궤적을 stepPx 간격으로 다시 뽑는다. 선분 길이를 따라가며 carry 를 이용해 다음 샘플 지점을 찾는다.

_circlePoints

원 둘레 길이를 구한 뒤 둘레를 충분한 개수의 점으로 나눈다.

_rectPoints

사각형의 네 변을 각각 선분 샘플링한 뒤 이어 붙인다.

_sampleSegment

점 a 와 b 사이 선분을 일정 간격으로 나눈다.

14. 표(DataTable)는 무엇을 보여주나?

표는 Current 또는 특정 Stroke 의 X, Y, Z 를 보여준다. Z 칸은 직접 수정할 수 있다. 한 칸의 값을 바꾸면 setState 안에서 해당 점의 zValues 가 갱신된다.

Current 와 Stroke 의 차이 아직 확정되지 않은 현재 도형을 보고 싶으면 Current, 이미 저장된 선 하나만 보고 싶으면 S0, S1 같은 stroke 를 선택한다.

15. 선택, 이동, 점 편집

기능	어떻게 켜짐	실제 동작
Select	FilterChip 선택	드래그 사각형과 겹치는 stroke 를 선택
Move	우클릭 드래그	선 전체를 delta 만큼 이동
Point Edit	Shift + 클릭/드래그	가장 가까운 샘플 점을 잡아 이동

이 기능들은 모두 “그리기”와는 다른 모드이기 때문에 _onPointerDown / Move / Up 의 가장 앞부분에서 먼저 검사한다. 즉 이동 중이면 새 선을 그리면 안 되고, 점 편집 중이면 선택 박스를 만들면 안 된다.

16. Smooth 슬라이더는 실제로 뭘 하나?

선택된 stroke 의 점들을 이동 평균으로 부드럽게 만든다. 윈도우 크기가 클수록 더 매끈해지지만 원래 모양이 더 많이 바뀔 수 있다. basePath 와 baseSamples 를 따로 보관해 두는 이유는, 스무딩을 여러 번 바꿀 때 이미 뭉개진 결과 위에 또 스무딩하지 않기 위해서다.

17. TCP 연결과 전송

16. _testConnection()은 _ensureConnected()를 호출해 연결만 시험한다.
17. _ensureConnected()는 host 와 port 를 읽고, 필요하면 기존 소켓을 끊고 새로 연결한다.
18. _selectSourceAddress()는 같은 대역의 IPv4 주소를 골라 sourceAddress 로 쓰려 한다.
19. 연결되면 socket.listen 으로 서버 응답을 계속 듣는다.
20. _sendJson()은 _buildPayload() 결과를 jsonEncode 해서 한 줄로 전송한다.

왜 을 붙일까? 서버가 한 줄 단위로 메시지를 읽는 경우가 많기 때문이다. JSON 뒤에 줄바꿈을 붙이면 메시지 경계를 알기 쉬워진다.

18. _buildPayload 를 자세히 보기

이 함수는 최종 전송 데이터의 모양을 만든다. 결과 형태는 대략 {"points": [[x, y, z], ...]} 이다. stroke 가 여러 개면 stroke 하나가 끝날 때마다 ["000"] 마커를 넣는다.

상황	결과
stroke 없음	현재 _pointsMm 와 _currentZValues 를 사용
stroke 있음	각 stroke 의 samples 를 image 좌표로 바꾸고 mm 변환 후 z 와 묶음
stroke 끝	["000"] 추가

마커의 의미 서버 쪽에서 “여기까지가 한 개의 stroke”라고 구분하기 위한 경계 표시다.

19. LinePainter 는 화면에 무엇을 그리나?

- 이미 저장된 모든 stroke path 를 선으로 그린다.
- 각 stroke 의 sample 점을 작은 원으로 그린다.
- 선택된 stroke 는 주황색으로 강조한다.
- 현재 그리고 있는 line, pen, circle, rect 도 임시로 보여준다.

- 각 점 옆에 번호를 붙인다.
- 선택 박스가 있으면 사각형 테두리를 그린다.

20. 코드 읽는 순서 추천

21. 먼저 변수 묶음을 읽어 전체 상태를 파악한다.
22. 다음으로 _onPointerDown, _onPointerMove, _onPointerUp 을 본다.
23. 그 다음 _rebuildPoints 와 샘플링 함수들을 본다.
24. 이후 _buildPayload, _sendJson, _ensureConnected 를 보면 전송 구조가 보인다.
25. 마지막으로 build 와 LinePainter 를 보면 화면이 왜 그렇게 보이는지 이해된다.

21. 자주 헷갈리는 부분 정리

헷갈리는 것	정리
_p0 / _p1	현재 그리고 있는 도형의 시작점/현재점
_rawPoints / _pointsPx	raw 는 원본 입력, pointsPx 는 가공된 샘플
_pointsImage / _pointsMm	하나는 프레임 기준 좌표, 하나는 mm 환산 좌표
path / samples	path 는 경로, samples 는 일정 간격 점들
_currentZValues / stroke.zValues	현재 미확정 도형용 / 저장된 stroke 용

22. 직접 수정해 볼 만한 포인트

- Continue 를 line 만 허용할지, pen 만 허용할지 규칙을 더 분명하게 만들기
- circle 과 rect 에서도 stroke 저장 시 z 초기값을 사용자가 지정한 값으로 넣기
- _applyTransform 에 회전과 yFlip 옵션을 UI 에서 바꿀 수 있게 만들기
- payload 마커 ["000"] 대신 더 명확한 구조를 쓰기. 예: strokes: [[[x,y,z],...], ...]
- 선택된 점 하나를 표에서 클릭해 캔버스에서 강조 표시하기

23. 마지막 요약

이 앱의 진짜 핵심은 “상태를 잘 나눠 저장하고, 포인터 이벤트마다 그 상태를 바꾸고, _rebuildPoints 로 실제 점 데이터를 다시 만드는 것”이다. 사용자는 그냥 선을 그리는 것처럼 느끼지만, 내부에서는 현재 위치, 이어 그리기 여부, 기존 stroke, 좌표 변환, 샘플링, Z 값, JSON 생성이 차례대로 맞물려 돌아간다. 특히 _onPointerDown 의 블록은 그리기의 출발 조건을 정하는 부분이라 반드시 이해해야 한다.

한 줄 결론 이 코드는 단순 그림판이 아니라, 그린 선을 기계가 이해할 수 있는 점 데이터로 바꾸는 도구다.